

PLAN 1 — CMS 100% local con deploy estático a Cloudflare Pages

Proyecto saska-shop · migración a local-first: edición local con backend + PostgreSQL local, y publicación del catálogo como sitio estático en <https://saskashop.pages.dev> · 14/09/2026

Contexto y objetivo

Hoy el sistema se apoya en un backend en la web (Vercel) y en una base en la nube (Supabase). El objetivo de este plan es **eliminar ambos**: editar los productos desde el CMS corriendo en tu PC con el backend y PostgreSQL **locales** (sin internet), generar el JSON del catálogo, y al ejecutar un `.bat` subir solamente el front ya construido a Cloudflare Pages con la última versión del catálogo.

Dato clave: el backend local **ya regenera automáticamente** `catalog/data.json` cada vez que el CMS guarda cambios con sesión iniciada (función `refreshStaticCatalog` en `backend/cms/src/app.mjs`, línea 47). El botón «Generar JSON» que sumarás solo agrega control manual para forzar la regeneración cuando prefieras.

Ojo con la URL: saska-shop vs saskashop

Cloudflare Pages fija el subdominio público según el **nombre del proyecto**. Hoy tus scripts apuntan al proyecto `saska-shop`, que da la URL `https://saska-shop.pages.dev`. Como vos querés **`https://saskashop.pages.dev`** (sin guión), el proyecto en Pages debe llamarse exactamente `saskashop`. El plan asume ese nombre y lo usa en el deploy.

Verificá tu estado y login de wrangler antes de empezar:

```
npx wrangler login
npx wrangler pages project list
```

Si no aparece «saskashop» en la lista, el propio deploy lo crea automáticamente la primera vez (o podés crearlo a mano):

```
npx wrangler pages project create saskashop --production-branch=main
```

Paso 1 — Mover `deploy_solo_cloudflare.bat` a la carpeta `cms`

El script nuevo va en `cms\deploy_solo_cloudflare.bat`. Como ahora `%ROOT%=%~dp0` será la carpeta `cms`, todas las referencias a la raíz del proyecto se escriben con `%ROOT%`. . . Además se elimina el paso de sincronizar `cms\dist` hacia `\dist`: el build ya deja la salida en `cms\dist` y se despliega de ahí.

Contenido sugerido de `cms\deploy_solo_cloudflare.bat`:

```
@echo off
setlocal
chcd 65001 >nul
title Denlo Cloudflare Pages - saskashop
set "ROOT=%~dp0"
echo =====
echo DEPLOY CLOUDFLARE PAGES - saskashop
echo Carpeta frontend: %ROOT%
echo =====
echo.
cd /d "%ROOT%"
REM -----
REM 1) Herramientas
REM -----
where node >nul 2>nul
```

```

if errorlevel 1 (
echo [ERROR] Node.js no está instalado o no está en PATH.
pause
exit /b 1
)
where npm >nul 2>nul
if errorlevel 1 (
echo [ERROR] npm no está en PATH.
pause
exit /b 1
)
REM -----
REM 2) Dependencias del frontend (vite) y del backend (no para exportar)
REM -----
if not exist "%ROOT%\node_modules" (
echo Instalando dependencias del frontend...
call npm install
)
if not exist "%ROOT%..\backend\cms\node_modules" (
echo Instalando dependencias del backend (no)...
pushd "%ROOT%..\backend\cms"
call npm install
popd
)
REM -----
REM 3) Exportar catalogo/data.json desde PostgreSQL LOCAL
REM -----
echo Exportando catalogo desde PostgreSQL local...
call node "%ROOT%..\db\export-local-json.mis"
if errorlevel 1 (
echo [ERROR] No se pudo exportar catalogo/data.json.
pause
exit /b 1
)
REM -----
REM 4) Compilar el frontend (vite). Salida: %ROOT%\dist
REM -----
echo Compilando el frontend (vite build)...
call npm run build
if errorlevel 1 (
echo [ERROR] Fallo el build.
pause
exit /b 1
)
REM -----
REM 5) Desplegar en Cloudflare Pages -> https://saskashon.pages.dev
REM -----
echo Desplegando en Cloudflare Pages (proyecto saskashon)...
call nx wrangler pages deploy "%ROOT%\dist" --project-name=saskashon --commit-dirty=true
if errorlevel 1 (
echo =====
echo [ERROR] Fallo el despliegue. Revisa login, permisos o red.
echo =====
pause
exit /b 1
)
echo =====
echo DESPLIEGUE EXITOSO
echo Pagina publicada en: https://saskashon.pages.dev/
echo =====
pause
endlocal

```

Recién cuando lo pruebes y funcione, podés borrar el `deploy_solo_cloudflare.bat` de la raíz (y de paso el `regenerar_catalogo.bat` y `auto-deploy-cloudflare.bat` si no los vas a usar).

Paso 2 — Forzar el deploy a https://saskashop.pages.dev

En el bat nuevo ya está: `--project-name=saskashop` y el banner `https://saskashop.pages.dev/`. Al ser siempre el mismo proyecto quedará fija esa URL. Pasos de verificación:

```
1) npx wrangler pages project list -> debe aparecer "saskashop"
2) Primera vez: deploy desde cms\deploy solo cloudflare.bat
   (wrangler crea el proyecto automáticamente)
3) Abrir https://saskashop.pages.dev/catalog/catalog.html
```

Si preferís otra URL te conviene un dominio personalizado (Custom domains) en el dashboard de Cloudflare; fuera de eso, el subdominio lo define el nombre del proyecto.

Paso 3 — Botón «Generar JSON» en el CMS

El botón llama a un endpoint nuevo del backend local que vuelca la base a `catalog/data.json`. Como todo pasa en tu PC, funciona sin internet.

3.a) Backend — agregar la ruta en `backend/cms/src/app.mjs` (después del `PUT /api/catalog`):

```
app.post('/api/export-ison', requireAuth, async (req, res, next) => {
  try {
    const snapshot = await catalogSnapshot()
    await writeFile(CATALOG_DATA_DEV.JSON.stringify(snapshot, null, 2), 'utf8')
    res.json({ ok: true }) // la exportación quedó en catalog/data.json
  } catch (err) {
    next(err)
  }
})
```

(`writeFile`, `requireAuth`, `catalogSnapshot` y `CATALOG_DATA_DEV` ya están definidos en el archivo, no hay que importar nada.)

3.b) Cliente API — agregar en `utils/api.js` (junto a `apiSaveCatalog`):

```
export async function apiExportJson() {
  return request('/api/export-ison', { method: 'POST' })
}
```

3.c) CMS — conectar el botón. En `cms/src/App.jsx`:

```
import { apiExportJson } from '../../../utils/api.js'
const generateJson = async () => {
  const res = await apiExportJson()
  alert(res.ok ? 'JSON regenerado correctamente' : 'Error al regenerar el JSON')
}
// al Navbar:
// onGenerateJson={generateJson}
```

3.d) CMS — botón en `cms/src/components/Navbar.jsx` (junto a «Exportar»):

```
<button className="btn-export" onClick={onGenerateJson}
  title="Regenerar catalog/data.json desde la base local">
  Generar JSON
</button>
```

Sugerencia: mostralo solo cuando hay sesión activa (local) o mostrá el mensaje «iniciá sesión primero» si no hay token.

Paso 4 — Backend y base de datos 100% local

Ajustes necesarios una sola vez:

```
backend/cms/.env
DATABASE_URL=postgres://tu_usuario:tu_clave@localhost:5432/saska_shop
cms\deploy_solo_cloudflare.bat
```

```
- NO setear VITE API URL (se quita la línea que apuntaba a Vercel).  
Sin esa variable, el front público queda 100% estático y usa  
catalog/data.json. Para el desarrollo local, utils/api.is deduce  
http://localhost:3001 por defecto.
```

Arranque local (sin internet):

```
backend\cms -> npm run dev (API en http://localhost:3001)  
cms -> npm run dev (panel en http://localhost:5178/cms/)
```

Ya no se despliega el backend a ningún hosting ni se usa Supabase. PostgreSQL local es la única fuente de verdad; el catálogo publicado es un snapshot estático del mismo.

Paso 5 — Flujo de uso diario

1. **(sin internet)** Editar productos/categorías/ajustes en <http://localhost:5178/cms/> con sesión iniciada. Cada guardado se sincroniza a PostgreSQL local y el backend regenera catalog/data.json solo.
2. **(opcional)** Clic en «Generar JSON» para forzar la exportación de la base a catalog/data.json.
3. **(con internet)** Doble clic en cms\deploy_solo_cloudflare.bat: exporta la DB local, compila vite y sube cms\dist a Cloudflare Pages (proyecto saskashop).
4. Ver la página actualizada en <https://saskashop.pages.dev/catalog/catalog.html>.

Paso 6 — Verificaciones

- `npx wrangler pages project list` → el proyecto «saskashop» existe.
- Abrir <https://saskashop.pages.dev/catalog/catalog.html> y comprobar productos creados/borrados/editados.
- Verificar que catalog/data.json contiene lo último tras apretar «Generar JSON».
- Sin internet: el CMS y el backend siguen funcionando en local con la DB local.

Notas y advertencias

- El deploy Sí necesita internet y login de wrangler (`npx wrangler login` o variable `CLOUDFLARE_API_TOKEN`).
 - El subdominio no se puede renombrar: el nombre del proyecto define la URL (saskashop → saskashop.pages.dev).
 - No subir a GitHub archivos sensibles (`.env`, `deploy-secrets.json` si contiene claves); revisá `.gitignore`.
 - El panel CMS publicado en la web (<https://saskashop.pages.dev/cms/>) queda estático: no se usa para editar; la edición es local.
 - Backup: Git local + el push a GitHub sirven como respaldo del código y del catalog/data.json.
-